A Synopsis

Of

Project

On

Probabilistic Anomaly Scoring for Explainable Digital Forensic Triage

Submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

Under the Guidance of

Dr. Rehan
Assistant Professor

Project Team ID: MP2026CSE75



Submitted By

Nitya Sanguri	**2023492**
Adarsh Bhandari	**2023980**

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
GRAPHIC ERA DEEMED TO BE UNIVERSITY
DEHRADUN

SEPTEMBER 2026

CERTIFICATE

This is to certify that the project synopsis entitled **"Probabilistic Anomaly Scoring for Explainable Digital Forensic Triage"** is being submitted by **Nitya Sanguri (2023492), Adarsh Bhandari (2023980)** for the partial fulfillment of the requirement for the award of the degree of **B. Tech-CSE** in Department of Computer Science & Engineering, Graphic Era Deemed to be University, Dehradun, and is a record of their own work carried out under my supervision. The matter embodied in the synopsis has not been submitted in any other university or institution for the award of any degree.

Dr. Rehan
Assistant Professor
Department of Computer Science and Engineering,
Graphic Era Deemed to be University, Dehradun, India.

DECLARATION

We hereby declare that the project synopsis entitled "**Probabilistic Anomaly Scoring for Explainable Digital Forensic Triage"** is being submitted for the partial fulfillment of the requirement for the award of the degree of **B. Tech-CSE** in Department of Computer Science & Engineering, Graphic Era Deemed to be University, Dehradun, is our original work and has not been submitted to any other university or institution for the award of any degree or diploma.

Nitya Sanguri **2023492**
Adarsh Bhandari **2023980**

TABLE OF CONTENTS

S. No.	Content	Page No.
	Abstract	**1**
1.	**Introduction**	**2**
1.1	Background	2
1.2	Motivation	2
1.3	Scope of the Project	3
2.	**Literature Review**	**4**
2.1	AI and Machine Learning in Digital Forensics	4
2.2	Explainable AI for Unsupervised Forensic Analysis	4
2.3	Supervised Work on CIC MalMemAnalysis-2022	5
2.4	Related Datasets	5
2.5	Research Gap	6
3.	**Problem Statement and Objectives**	**7**
3.1	Problem Statement	7
3.2	Objectives	7
3.3	Scope and Limitations	8
4.	**Proposed Methodology**	**9**
4.1	Phase 1 — Dataset Acquisition and Auditing	9
4.2	Phase 2 — Preprocessing and Group-Aware Splitting	9
4.3	Phase 3 — Probabilistic Anomaly Model	10
4.4	Phase 4 — Evaluation	10
4.5	Phase 5 — Explanation and Explanation Audit	10
4.6	Phase 6 — Cross-Modality Replication and Triage Output	11
5.	**Hardware and Software Requirements**	**13**
5.1	Hardware Requirements	13
5.2	Software Requirements	14
5.3	Dataset Handling	15
6.	**Expected Outcomes**	**17**
6.1	A Corrected Baseline	17
6.2	A Probabilistic Triage Score	17
6.3	An Explanation Audit	17
6.4	Dataset Audit Results	17
6.5	Deliverables	18
7.	**Conclusion**	**19**
	References	**20**

ABSTRACT

Digital forensic examiners routinely face storage volumes and memory images far larger than can be reviewed by hand, and the machine learning tools proposed to triage that material are usually supervised, opaque, and evaluated only on aggregate accuracy — properties that are problematic in forensics, where labelled evidence is rarely available at the point of seizure and an opinion resting on a model that cannot explain itself is difficult to defend under evidentiary scrutiny. This project proposes a probabilistic anomaly scoring framework for explainable digital forensic triage. Rather than issuing a binary benign-or-malicious verdict, the system trains generative unsupervised models — a variational autoencoder and a deterministic autoencoder — on benign memory states alone, and assigns every artefact a likelihood-based anomaly score, calibrated on held-out benign data into a p-value (conformal calibration), so that an examiner can flag artefacts against an explicit false-alarm budget. Shapley Additive Explanations (SHAP) are then applied to those scores to identify which memory features drove each decision, and the explanations are audited against trivial single-feature baselines to test whether the model is right for the right reasons. The work extends Khalid et al. [1], which combined Isolation Forest, autoencoder and clustering methods with SHAP on the CIC MalMemAnalysis-2022 dataset, in three ways: a probabilistic score in place of a fixed reconstruction cut-off, group-aware evaluation that prevents dumps of the same binary appearing in both training and test partitions, and reporting of precision, recall, F1 and PR-AUC for the anomaly class rather than accuracy alone. The framework is validated on CIC MalMemAnalysis-2022, with CICIoT2023 (network flows) and BODMAS (Windows executables with first-seen dates) used as cross-modality and temporal checks; Dumpware10 is retained only for a duplicate audit.

CHAPTER 1

INTRODUCTION

1.1 Background

Digital forensics is the discipline of acquiring, preserving and analysing digital evidence in a manner that survives legal challenge. Its central operational problem is volume: a single seized workstation can yield terabytes of disk content and gigabytes of volatile memory, and an examiner must decide within hours which artefacts merit deeper analysis. This decision is called triage. Dunsin et al. [2] survey the application of artificial intelligence and machine learning across the digital forensics and incident response lifecycle and conclude that, while AI materially improves throughput in data recovery, timeline reconstruction and pattern recognition, its adoption is constrained by a lack of standardisation and by persistent interpretability problems.

Memory forensics is a particularly acute case. Obfuscated, packed and fileless malware may leave no meaningful artefact on disk while remaining fully resident in RAM. The CIC MalMemAnalysis-2022 dataset was constructed for exactly this problem, using the VolMemLyzer-V2 tool to extract 55 numeric features from memory dumps of ransomware, spyware and trojan families alongside benign captures [3].

1.2 Motivation

Two gaps motivate this project. First, almost all published work on forensic memory datasets is supervised, which presumes labelled evidence that does not exist at the point of seizure and cannot generalise to malware families absent from the training set. Khalid et al. [1] argue that unsupervised learning is the practically relevant paradigm for forensics precisely because labelling large forensic images is expensive, expert-dependent and introduces its own biases.

Second, the outputs currently produced are the wrong shape for legal use. A model that returns “malicious” with 99% reported accuracy offers an examiner no way to express uncertainty, no way to tune a false-alarm rate to the resources of a given investigation, and no defensible account of why a particular artefact was flagged. Under evidentiary standards governing expert testimony, an opinion derived from a black-box process is weak. What an examiner needs instead is a calibrated score with a stated error rate and an explanation that can be interrogated.

Equally important, and largely unaddressed in the literature, is whether the explanations themselves are trustworthy. Preliminary analysis of CIC MalMemAnalysis-2022 indicates that the benign dumps were captured on a near-idle machine: across 29,298 benign rows the running-services count takes only eight distinct values, and a single decision stump on that one feature reaches 99.66% accuracy. The features that published SHAP rankings surface as most important are largely the same capture-environment features. An explanation that looks convincing while describing an artefact of the acquisition setup rather than malicious behaviour is more dangerous in court than no explanation at all.

1.3 Scope of the Project

This project designs, implements and evaluates a probabilistic anomaly scoring framework for forensic triage, trained on benign artefacts only, producing a continuous likelihood-based score, and accompanied by an explanation layer that is itself audited against trivial baselines. The framework is developed primarily on CIC MalMemAnalysis-2022 and then repeated on CICIoT2023 (network telemetry) and BODMAS [22] (static features of Windows executables) to test whether the protocol transfers across evidence modalities. BODMAS is the harder and more realistic benchmark: its 134,435 executables span 581 real-world malware families and a broad population of benign software, no single feature separates the classes (the best one-feature rule reaches 78.4%, against 99.66% on CIC MalMemAnalysis-2022), and every file carries a first-seen date, so a model can be tested on files newer than any it was trained on. Dumpware10 (memory dumps rendered as images) was dropped from modelling after detectors trained on benign images alone caught at most about 5% of malware at a 1% false-alarm rate, although a supervised model separates the classes; it is retained for its duplicate audit. The project does not attempt to build a courtroom-ready product; it aims to establish an evaluation protocol and a scoring output that are honest about their own limitations.

CHAPTER 2

LITERATURE REVIEW

2.1 AI and Machine Learning in Digital Forensics

Dunsin et al. [2] provide the most comprehensive recent analysis of AI and ML in digital forensics and incident response, covering data collection and recovery, cybercrime timeline reconstruction, big data analysis, pattern recognition, chain-of-custody protection and automated incident response. Their conclusion is that AI offers clear efficiency gains but that growing evidence volumes, evolving criminal tradecraft, a lack of standardisation and unresolved interpretability concerns limit operational deployment. Explainability is identified as a prerequisite rather than an enhancement.

2.2 Explainable AI for Unsupervised Forensic Analysis

The base paper for this project, Khalid, Iqbal and Saqib [1], addresses the observation that XAI research has concentrated almost entirely on supervised models, whereas forensic data is typically unlabelled. Using CIC MalMemAnalysis-2022, they apply Isolation Forest and an autoencoder for anomaly detection, and K-means, DBSCAN and a Gaussian Mixture Model for clustering at the binary, 4-class and 16-class levels, explaining the results with SHAP force, waterfall, scatter, summary and bar plots. To create an anomaly-detection setting they retain only the first 5% of malicious instances. The reported results are: Isolation Forest 0.8595 accuracy; autoencoder 0.9759 accuracy; K-means binary ARI 0.7544; DBSCAN at the 16-class level ARI 0.5690, discovering only 9 clusters; GMM binary ARI 0.9592; and GMM 4-class ARI 0.6449. The SHAP rankings surface handles.nsection, svcscan.nactive, pslist.avg_handlers and ldrmodules.not_in_load among the most influential features.

Earlier approaches to explaining unsupervised models are largely indirect. Montavon et al. [4] convert an unsupervised model into an equivalent neural network before applying layer-wise relevance propagation, and Morichetta et al. [5] feed cluster assignments into a supervised classifier as pseudo-labels before applying LIME — an approach whose authors acknowledge introduces bias. SHAP applied directly to unsupervised scores, as in Khalid et al. [1], avoids that intermediate step.

2.3 Supervised Work on CIC MalMemAnalysis-2022

The dataset has been worked heavily, and almost entirely in a supervised setting. Carrier et al. [3] report 99% with a stacking ensemble; Dener et al. [6] reach 99.97% with logistic regression; Mezina and Burget [7] obtain about 99% binary accuracy with Random Forest and 83% on multi-class classification with a dilated CNN; Öztürk and Hızal [8] report 99.99%, 87.79% and 75.49% for binary, 4-class and 16-class respectively using XGBoost. Roy et al. [9] and Shafin et al. [10] report the same pattern — near-perfect binary results collapsing to roughly 85% at 4 classes and 67–73% at 16 classes. Autoencoder work reports near-perfect figures: Dai et al. [11] feed the reconstruction error of an autoencoder trained on benign dumps into a supervised classifier and report 99.99% accuracy and F1 on a random 80/20 split, while themselves noting a possible overfitting problem, and Fuhnwi et al. [12] combine autoencoder features with logistic regression for 99.97% accuracy and a 99.98% detection rate. Both use malware labels in training, so neither tests a score learned from benign data alone. Unsupervised and explainable work is thin: a one-class support vector machine appears once [13], achieving F1 of only 0.44 on raw features before principal component analysis rescues it, and XMal [14] keeps five of the 55 features, selected by recursive feature elimination, and uses SHAP to explain the resulting classifier, with accuracy above 99%.

2.4 Related Datasets

Dumpware10 [15] renders memory dumps of 10 malware families and benign software (11 classes) as images, with GIST and HOG features and an SMO classifier reaching 96.39%; Shah et al. report 97.01% and 97.82% with support vector machines on denoised versions of the images [16], [21]. Explainability work on it is minimal and instrumental — LIME used to crop regions before classification [17], Grad-CAM used to design kernels — rather than auditing whether the model is correct for the right reasons. For network evidence, Neto et al. [18] introduce CICIoT2023 and report Random Forest at 0.997 binary but only 0.714 macro-F1 across 34 classes; subsequent work frequently reports headline accuracy while omitting per-class recall, with benign-class F1 as low as 73% behind a 98.42% headline. Re-extracting the raw captures with a time-based flow exporter [19] gives very different counts per class from the released files (595,785 benign flows against 1,098,191 released rows), because the released rows are fixed-size packet windows rather than flows; the published feature files therefore reflect one aggregation choice rather than the traffic itself. For executable evidence, BODMAS [22] provides 2,381 static features of 57,293 malware

and 77,142 benign Windows executables, with malware first seen between August 2019 and September 2020, and was released for temporal analysis of malware detection.

2.5 Research Gap

Synthesising the above: we found no study that applies a variational autoencoder to these datasets or reports anomaly-class metrics for a score learned from benign data alone; the autoencoder studies that exist pair the autoencoder with a supervised classifier; group-aware splitting on the malware hash is rarely performed, despite each binary contributing ten dumps taken fifteen seconds apart [3]; and we found no study that tests whether SHAP explanations on these datasets describe malicious behaviour or capture-environment artefacts. This project targets those four gaps directly.

CHAPTER 3

PROBLEM STATEMENT AND OBJECTIVES

3.1 Problem Statement

Existing machine learning support for digital forensic triage is supervised, opaque and evaluated in a way that overstates its reliability. Models depend on labelled evidence that is unavailable at the point of seizure; they emit binary verdicts with no calibrated measure of uncertainty; their reported accuracy is inflated by dataset artefacts including synthetic class balancing, near-idle benign captures and random splits that place multiple dumps of the same binary on both sides of the train–test boundary; and the explanations produced for them have never been audited to confirm that they describe malicious behaviour rather than properties of the acquisition environment. Consequently an examiner has no defensible basis on which to act on, or testify to, the output of such a system.

3.2 Objectives

1. To train a variational autoencoder and a deterministic autoencoder on benign artefacts only, scoring each unseen artefact by its likelihood under the learned benign distribution and converting the score into a p-value on held-out benign data, so that the false-alarm rate is fixed in advance.
2. To report precision, recall, F1 and PR-AUC for the anomaly class against majority-class and single-feature stump baselines.
3. To split on the sample SHA-256 hash so all dumps of one binary stay on one side, and quantify the gap against random splits.
4. To audit SHAP explanations against the single-feature stump ranking, testing whether they reflect malicious behaviour or capture-environment artefacts.
5. To repeat the protocol on CICIoT2023, with a duplicate audit and the packet-window confound check, and on BODMAS, training on executables first seen before March 2020 and testing on later ones; and to audit Dumpware10 for duplicate images.
6. To output a precision–recall curve and ranked artefact list, letting the examiner set the false-alarm budget explicitly.

3.3 Scope and Limitations

The evaluation is confined to three public datasets. CIC MalMemAnalysis-2022 never recorded timestamps and CICIoT2023 has them removed, so temporal generalisation — whether a model trained on today's data holds next year — can be tested only on BODMAS, whose malware spans a single year (August 2019 to September 2020). That is a short horizon on one evidence type, and this constraint is stated openly rather than left for a reviewer to discover.

CHAPTER 4

PROPOSED METHODOLOGY

The methodology proceeds in six phases, applied first to CIC MalMemAnalysis-2022 and then repeated on CICIoT2023 and BODMAS.

4.1 Phase 1 — Dataset Acquisition and Auditing

The three public datasets are acquired and audited before any model is trained. The audit records class composition, exact-duplicate rate, the number of distinct values taken by each feature, and any evidence that a single feature separates the classes on its own. For CIC MalMemAnalysis-2022 this includes the dataset authors' statement that the 50/50 balance was reached by oversampling the benign class with SMOTE [3, p. 184], meaning that an unreported share of the benign class may be synthesised rather than observed; the audit looks for traces of interpolation, such as fractional values in features that are whole-number counts (none were found in the 40 count features, so any synthetic rows were rounded). For Dumpware10 the audit uses perceptual hashing of the images to establish per-family duplicate and near-duplicate rates, the first systematic duplicate audit of the dataset that we are aware of. For CICIoT2023 it includes measuring duplication across all 63 merged files and quantifying the packet-window confound, where the window size is 10 packets for benign and low-volume traffic but 100 for flood attacks. For BODMAS it includes the first-seen span of each class, family composition and feature scale.

4.2 Phase 2 — Preprocessing and Group-Aware Splitting

Ground-truth labels are removed from the training pipeline and retained only for evaluation. Duplicate rows are dropped, and benign rows are split 60/20/20 into training, calibration and test partitions, fixed once and shared by every model; features are standardised with a StandardScaler fitted on the training partition only. The train–test split is constructed on the group key rather than at random: for CIC MalMemAnalysis-2022 the SHA-256 hash in the sample identifier, so that all ten dumps of a binary fall on one side; for BODMAS the first-seen date, so that models are tested on executables newer than any they were trained on. A random split is also generated for both, so the inflation attributable to leakage can be reported as a number rather than asserted.

4.3 Phase 3 — Probabilistic Anomaly Model

A variational autoencoder is trained on benign artefacts only. The encoder maps an artefact to a distribution over a low-dimensional latent space; the decoder reconstructs from a sample of that distribution; training minimises reconstruction error together with the KL divergence between the approximate posterior and a standard normal prior. At inference the negative evidence lower bound serves as a probabilistic anomaly score: artefacts poorly explained by the benign distribution receive high scores. A deterministic autoencoder and an Isolation Forest are trained under identical conditions as comparators, reproducing the configuration of the base study. Each score s is converted into a conformal p-value, $p = (1 + \text{number of calibration scores} \geq s) / (n + 1)$, over the n held-out benign calibration rows; flagging at $p \leq 0.01$ keeps false alarms on unseen benign data at about 1%, and only after this step is the score described as calibrated.

4.4 Phase 4 — Evaluation

Every model is evaluated with a confusion matrix and with precision, recall, F1 and PR-AUC computed for the anomaly class, supported by a full precision–recall curve so that an examiner can select an operating point against a chosen false-alarm budget. Accuracy is reported only alongside the two trivial baselines — majority-class prediction and a one-feature decision stump — so that any apparent gain is visible as a gain over what a trivial rule already achieves. Clustering comparators are scored by Adjusted Rand Index against the reference labels, matching the base paper so that results are directly comparable.

4.5 Phase 5 — Explanation and Explanation Audit

SHAP [20] is applied to the anomaly score using the model-agnostic Explainer, producing local explanations (force and waterfall plots) for individual flagged artefacts and global explanations (summary and bar plots) across the corpus. The audit step is the novel contribution: the global SHAP ranking is placed alongside the ranking of features by single-stump discriminative power and alongside the leave-one-family-out results. Where the two rankings coincide on capture-environment features such as the running-services count, the explanation is documented as reflecting the acquisition setup rather than malicious behaviour.

4.6 Phase 6 — Cross-Modality Replication and Triage Output

The complete protocol is repeated on CICIoT2023 and BODMAS. For CICIoT2023 an ablation is run with and without the window-sensitive features (Number, Tot sum, IAT, Variance, Rate); the performance lost when they are removed measures how much of the model's apparent skill came from reading the feature-extraction setting. For BODMAS the models are trained and calibrated on executables first seen before March 2020 and tested on those first seen later, measuring whether the false-alarm rate and the detection rate survive the passage of time. The final deliverable is a triage report: artefacts ranked by probabilistic anomaly score, each with its top contributing features, its position on the precision–recall curve, and an explicit statement of the protocol's known limitations.

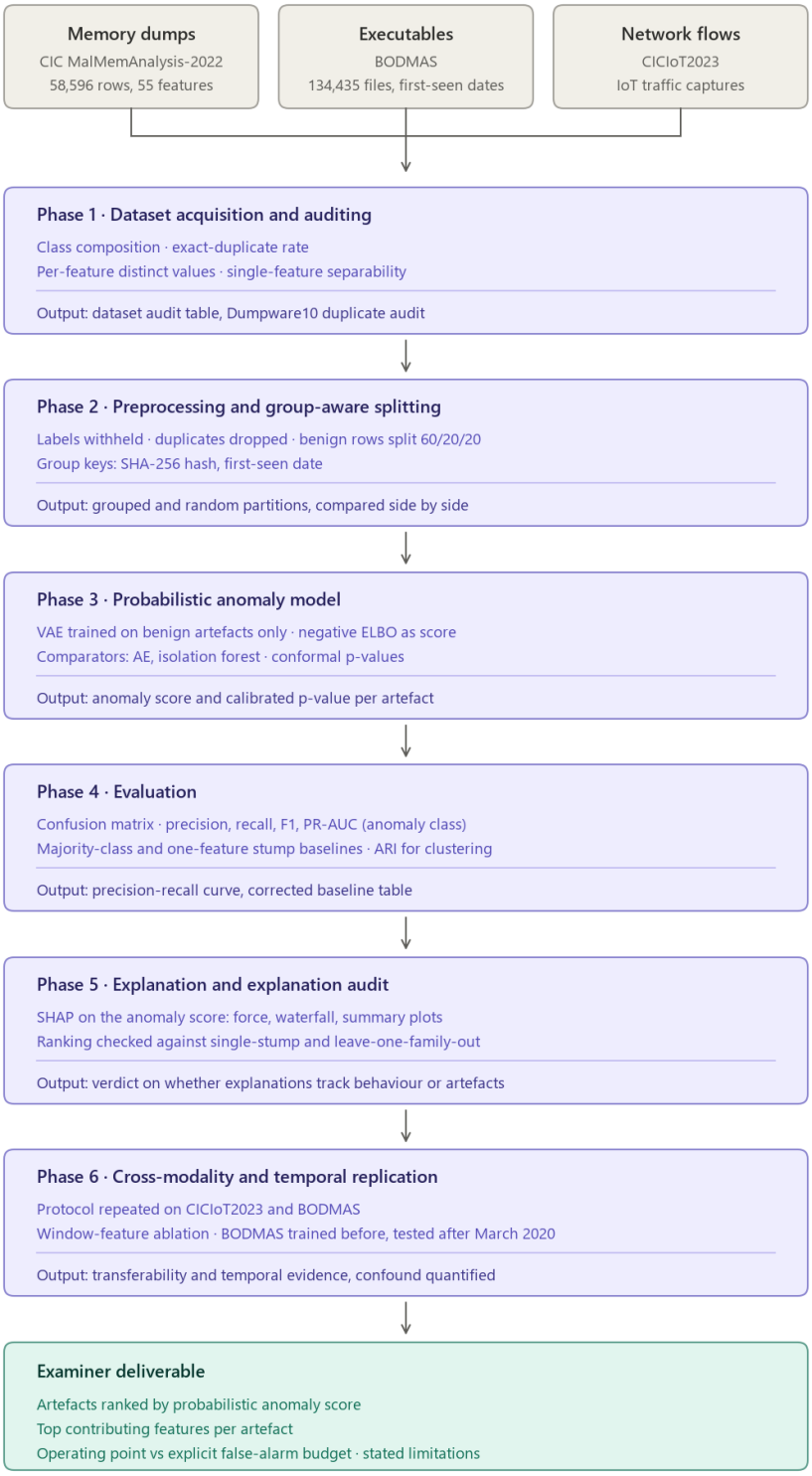


Fig. 4.1 Methodology pipeline: three evidence modalities — memory dumps, executables and network flows — feed six phases of auditing, group-aware splitting, probabilistic modelling, evaluation, explanation audit and cross-modality and temporal replication, producing the examiner deliverable

CHAPTER 5

HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Hardware Requirements

The study relies on three categories of data: network traffic captures and the flow features derived from them (CICIoT2023), memory-image snapshots and the VolMemLyzer feature tables extracted from them (CIC MalMemAnalysis-2022), and static features of Windows executables (BODMAS). Raw captures and memory dumps in these corpora range from several gigabytes to hundreds of gigabytes per sample set, and parsing them — together with training probabilistic models on the resulting features — is memory-bound rather than purely compute-bound. The configuration below is sized accordingly.

Table 5.1 Hardware configuration for dataset parsing, model training and evaluation

Component	Minimum	Recommended	Rationale
Processor	Intel Core i5 (10th Gen) / AMD Ryzen 5	Intel Core i7/i9 (12th Gen+) / AMD Ryzen 7/9	Multi-core throughput speeds up parallel feature extraction across large batches of packet captures and memory images.
RAM	16 GB	32–64 GB (DDR4/DDR5)	Memory-image parsing (Volatility) and matrix-based probabilistic inference both require the active working set to fit in memory; 16 GB is workable for subsets but becomes a bottleneck at full dataset scale.
Storage	512 GB SSD	1–2 TB NVMe SSD	Raw corpora, extracted feature tables, container images and virtual environments together occupy several hundred GB; NVMe throughput materially reduces I/O wait during parsing and training.
GPU	NVIDIA GTX 1660 (4 GB VRAM)	NVIDIA RTX 3060/4060 or higher (8 GB+ VRAM, CUDA-enabled)	Required only for the neural components — the variational and deterministic autoencoders. Classical baselines (Gaussian Mixture Models, Isolation Forest, PyMC inference on tabular features) run adequately on CPU, so a discrete GPU accelerates part of the pipeline rather than being a hard prerequisite for all of it.
Network	Standard broadband	High-speed broadband / fibre	Dataset corpora and containerised tool images are frequently 50–200+ GB per download; connection speed affects setup and iteration time rather than model quality.

Raw datasets are stored on a volume separate from the operating system and development environment, both to avoid contention with system I/O and to keep the raw/processed data boundary clean for reproducibility.

Where local hardware is insufficient, the pipeline is designed to run unmodified on cloud compute — Google Colab Pro, AWS EC2 GPU instances or Kaggle Kernels — with no code changes beyond environment configuration.

5.2 Software Requirements

5.2.1 Operating System and Environment

- Ubuntu 22.04 LTS (preferred, for native command-line integration with memory- and network-forensics utilities) or Windows 11 with WSL2.
- Docker and Docker Compose, for isolating forensic tools and ensuring reproducible builds across machines.
- Git, with GitHub or GitLab, for version control and experiment tracking.
- Python 3.10 or later, managed via venv or Conda for dependency isolation.

5.2.2 Languages and Development Tools

- Python — primary implementation language, used across preprocessing, modelling and evaluation.
- SQL — querying of extracted artefact and metadata stores.
- JavaScript / Node.js (optional) — only if a web-based results and visualisation dashboard is built.
- JupyterLab — exploratory data analysis and iterative feature investigation.
- VS Code or PyCharm — primary development environment for pipeline code.

5.2.3 Forensic Acquisition and Parsing

Table 5.2 Forensic acquisition and parsing tools used in the preprocessing layer

Tool	Role in the pipeline
Volatility 3	Memory forensics framework; extracts OS- and process-level artefacts from CIC MalMemAnalysis-2022 snapshots.
VolMemLyzer-V2	Converts Volatility output into the 55 numeric memory features used by the dataset and by this project.
Zeek	Generates structured connection-level metadata from CICIoT2023 traffic captures.
Suricata	Signature and anomaly-based inspection of network traffic; supplementary event generation.
Wireshark / TShark	Packet-level inspection and protocol analysis of PCAP files.
Scapy	Programmatic packet parsing and custom feature construction.

Tool	Role in the pipeline
dpkt	Lightweight, low-overhead PCAP parsing within Python pipelines.

These tools form the preprocessing layer that converts raw captures and memory images into structured, tabular features suitable for statistical and probabilistic modelling.

5.2.4 Probabilistic Modelling and Machine Learning Stack

- PyMC, TensorFlow Probability or GPyTorch — Bayesian inference, posterior estimation and uncertainty quantification, central to the probabilistic anomaly-scoring approach.
- PyTorch or TensorFlow with Keras — implementation of the variational autoencoder and the deterministic autoencoder.
- Scikit-learn — baseline detectors (Isolation Forest, One-Class SVM, Gaussian Mixture Models, K-means, DBSCAN) together with PCA, StandardScaler and the evaluation metrics used as comparison points against the probabilistic methods.
- SHAP — model-agnostic Explainer, producing the force, waterfall, scatter, summary and bar plots used in the explanation audit.
- Pandas and NumPy — core numerical and tabular data handling; SciPy for statistical testing.
- Pillow — image loading for the Dumpware10 duplicate audit.
- Dask — out-of-core and parallel processing for feature tables that exceed available system memory.
- Matplotlib and Seaborn — precision–recall curves, confusion matrices and figure generation.

5.2.5 Data Storage and Formats

- Apache Parquet / Feather — primary storage format for extracted features, chosen for columnar compression and fast partial reads at scale.
- CSV — small intermediate outputs and compatibility with external tools.
- PostgreSQL or MongoDB (optional) — persistent, queryable store for extracted artefacts, feature metadata and computed anomaly scores, where results need to be searchable rather than file-based.

5.3 Dataset Handling

- CIC MalMemAnalysis-2022 — memory-image corpus, used for process- and OS-artefact features; 58,596 records described by 55 VolMemLyzer-V2 features.

- BODMAS — static features (2,381 EMBER features) of 134,435 Windows executables, 57,293 of them malware from 581 families, each with a first-seen date; used for the cross-modality and temporal tests.
- CICIoT2023 — network traffic corpus, used for flow- and packet-level intrusion features and for the packet-window ablation.
- Dumpware10 — image-rendered memory dumps of 10 malware families and benign software, used only for the duplicate audit.
- Supplementary PCAP files and memory dumps, where used for additional validation beyond the three primary corpora.

Raw data is retained separately from processed output. Feature-extraction scripts produce a standardised, versioned feature table so that the same processed dataset can be reused consistently across the anomaly-detection models under comparison, rather than being re-derived per experiment. Together with the group keys recorded at extraction time — the SHA-256 hash for CIC MalMemAnalysis-2022 and the first-seen date for BODMAS — this ensures that every model in the comparison is trained and evaluated on identical partitions.

CHAPTER 6

EXPECTED OUTCOMES

The project is expected to produce the following outcomes.

6.1 A Corrected Baseline

A reproduction of the base study's Isolation Forest and autoencoder experiments on CIC MalMemAnalysis-2022, reported with confusion matrices, precision, recall, F1 and PR-AUC for the anomaly class, under both random and group-aware splits. Based on the preliminary audit, the group-aware split is expected to cost roughly 4 macro-F1 points at the 4-class level and roughly 10 points at the 16-class level relative to the random split — a quantified measure of the leakage present in previously published numbers.

6.2 A Probabilistic Triage Score

A variational autoencoder trained on benign artefacts alone that outputs a likelihood-based anomaly score, calibrated into a conformal p-value on held-out benign data and delivered with a precision–recall curve rather than a single accuracy figure. The expectation is explicitly not to beat published accuracy: on a dataset that is 50% malicious after synthetic balancing, accuracy is not a winnable or meaningful target. The contribution is an output an examiner can act on, with a false-alarm rate they choose.

6.3 An Explanation Audit

A side-by-side comparison of SHAP feature rankings, single-feature stump rankings and leave-one-family-out results. The anticipated finding is that the features SHAP identifies as most important on CIC MalMemAnalysis-2022 substantially overlap with the capture-environment features that already separate the classes on their own, demonstrating that a convincing-looking explanation can describe the acquisition setup rather than malicious behaviour. For a project concerned with the admissibility of AI-assisted findings, this is a more consequential result than a further feature-importance plot.

6.4 Dataset Audit Results

A per-family duplicate and near-duplicate rate for Dumpware10 — the first systematic duplicate audit of the dataset that we are aware of — together with the accuracy of a rule

that uses no learning at all, as the floor any published classifier must beat; a verified duplication figure for CICIoT2023 measured across all 63 merged files; a quantification of the label-dependent packet-window confound through an ablation on the window-sensitive features; and, for BODMAS, the change in false-alarm and detection rates between executables from the training period and those first seen after it.

6.5 Deliverables

A reproducible codebase and documented experimental protocol, a set of comparison tables and figures suitable for publication, and a triage report format that ranks artefacts by anomaly score with per-artefact explanations and a stated error rate.

CHAPTER 7

CONCLUSION

Digital forensic triage needs machine learning that works without labels, produces a score rather than a verdict, and can explain itself in terms an examiner is willing to defend; the current literature satisfies none of these conditions consistently, being overwhelmingly supervised, reporting aggregate accuracy on datasets whose construction inflates that accuracy, and presenting explanations rather than testing them. This project proposes a probabilistic anomaly scoring framework that trains a variational autoencoder on benign artefacts only, scores each unseen artefact by its likelihood under the learned benign distribution, and reports that score as a precision–recall curve against which an examiner can set an explicit false-alarm budget. It extends the unsupervised explainable-AI work of Khalid et al. [1] on CIC MalMemAnalysis-2022 in three directions — a probabilistic score in place of a fixed reconstruction threshold, group-aware evaluation on the malware hash, and anomaly-class metrics in place of aggregate accuracy — and repeats the protocol on CICIoT2023 and BODMAS to test whether it holds across evidence modalities and over time. Its most distinctive contribution is the explanation audit: establishing whether SHAP rankings on these benchmarks describe malicious behaviour or merely the conditions under which the evidence was captured is a precondition for any claim that AI-assisted forensic findings are explainable in a legally meaningful sense. Reporting that honestly — including where the result is unflattering, and including the limitation that only BODMAS carries timestamps, so temporal generalisation is tested on one dataset over a single year — is the standard this project sets for itself.

REFERENCES

[1] Z. Khalid, F. Iqbal, and M. Saqib, “Bridging knowledge gaps in digital forensics using unsupervised explainable AI,” Forensic Sci. Int.: Digit. Investig., vol. 53, art. no. 301924, 2025, doi: 10.1016/j.fsidi.2025.301924.

[2] D. Dunsin, M. C. Ghanem, K. Ouazzane, and V. Vassilev, “A comprehensive analysis of the role of artificial intelligence and machine learning in modern digital forensics and incident response,” Forensic Sci. Int.: Digit. Investig., vol. 48, art. no. 301675, 2024, doi: 10.1016/j.fsidi.2023.301675.

[3] T. Carrier, P. Victor, A. Tekeoglu, and A. H. Lashkari, “Detecting obfuscated malware using memory feature engineering,” in Proc. 8th Int. Conf. Information Systems Security and Privacy (ICISSP), 2022, pp. 177–188, doi: 10.5220/0010908200003120.

[4] G. Montavon, J. Kauffmann, W. Samek, and K.-R. Müller, “Explaining the predictions of unsupervised learning models,” in xxAI – Beyond Explainable AI, Lecture Notes in Computer Science, vol. 13200, 2022, pp. 117–138.

[5] A. Morichetta, P. Casas, and M. Mellia, “EXPLAIN-IT: Towards explainable AI for unsupervised network traffic analysis,” in Proc. 3rd ACM CoNEXT Workshop on Big Data, Machine Learning and Artificial Intelligence for Data Communication Networks, Orlando, FL, Dec. 2019, pp. 22–28.

[6] M. Dener, G. Ok, and A. Orman, “Malware detection using memory analysis data in big data environment,” Applied Sciences, vol. 12, no. 17, art. no. 8604, Aug. 2022.

[7] A. Mezina and R. Burget, “Obfuscated malware detection using dilated convolutional network,” in Proc. 14th Int. Congr. Ultra Modern Telecommunications and Control Systems (ICUMT), 2022, pp. 110–115.

[8] A. Öztürk and S. Hızal, “Detection and analysis of malicious software using machine learning models,” Sakarya University Journal of Computer and Information Sciences, vol. 7, no. 2, pp. 264–276, 2024.

[9] K. S. Roy, T. Ahmed, P. B. Udas, M. E. Karim, and S. Majumdar, “MalHyStack: A hybrid stacked ensemble learning framework with feature engineering schemes for obfuscated malware analysis,” Intelligent Systems with Applications, vol. 20, art. no. 200283, Nov. 2023.

[10] S. S. Shafin, G. Karmakar, and I. Mareels, “Obfuscated memory malware detection in resource-constrained IoT devices for smart city applications,” Sensors, vol. 23, no. 11, art. no. 5348, 2023.

[11] Z. Dai, L. Y. Por, Y.-L. Chen, J. Yang, C. S. Ku, R. Alizadehsani, and P. Pławiak, “An intrusion detection model to detect zero-day attacks in unseen data using machine learning,” PLOS ONE, vol. 19, no. 9, art. no. e0308469, 2024, doi: 10.1371/journal.pone.0308469.

[12] G. S. Fuhnwi, M. Revelle, and C. Izurieta, “A hybrid anomaly detection approach for obfuscated malware,” in Proc. IEEE Int. Conf. Cyber Security and Resilience (CSR), 2024, pp. 159–165, doi: 10.1109/CSR61664.2024.10679474.

[13] M. Al-Qudah, Z. Ashi, M. Alnabhan, and Q. Abu Al-Haija, “Effective one-class classifier model for memory dump malware detection,” Journal of Sensor and Actuator Networks, vol. 12, no. 1, art. no. 5, 2023, doi: 10.3390/jsan12010005.

[14] M. M. Alani, A. Mashatan, and A. Miri, “XMal: A lightweight memory-based explainable obfuscated-malware detector,” Computers & Security, vol. 133, art. no. 103409, 2023, doi: 10.1016/j.cose.2023.103409.

[15] A. S. Bozkir, E. Tahillioglu, M. Aydos, and I. Kara, “Catch them alive: A malware detection approach through memory forensics, manifold learning and computer vision,” Computers & Security, vol. 103, art. no. 102166, Apr. 2021.

[16] S. S. H. Shah, A. R. Ahmad, N. Jamil, and A. ur R. Khan, “Memory forensics-based malware detection using computer vision and machine learning,” Electronics, vol. 11, no. 16, art. no. 2579, 2022, doi: 10.3390/electronics11162579.

[17] Q. M. Yaseen, E. Oudat, M. Aldwairi, and S. Fraihat, “Efficient image-based memory forensics for fileless malware detection using texture descriptors and LIME-guided deep learning,” Computers, vol. 14, no. 11, art. no. 467, 2025, doi: 10.3390/computers14110467.

[18] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, “CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment,” Sensors, vol. 23, no. 13, art. no. 5941, 2023, doi: 10.3390/s23135941.

[19] J. Vitorino, D. Pinto, E. Maia, I. Amorim, and I. Praça, “Revisiting network traffic analysis: Compatible network flows for ML models,” in Foundations and Practice

of Security (FPS 2025), Lecture Notes in Computer Science, Springer, 2026, pp. 150–166, doi: 10.1007/978-3-032-20026-6_9.

[20] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 4765–4774.

[21] S. S. H. Shah, N. Jamil, and A. ur R. Khan, “Memory visualization-based malware detection technique,” Sensors, vol. 22, no. 19, art. no. 7611, 2022, doi: 10.3390/s22197611.

[22] L. Yang, A. Ciptadi, I. Laziuk, A. Ahmadzadeh, and G. Wang, “BODMAS: An open dataset for learning based temporal analysis of PE malware,” in Proc. 4th Deep Learning and Security Workshop (DLS), 2021.